

1. はじめに

このレポートの目的は、ソケット通信とスレッドを用いた並行プログラミングによって、2つの銀行間で取引データを処理するシステムを構築し、その動作と開発過程を記録することである。

特に、ファイルから読み込んだトランザクションをスレッドで並行処理し、他の銀行への送金はソケット通信を使って送金・受信する点が特徴である。

2. 組み込み機器の概要

raspberry pi の起動方法、プログラムのファイルを送り、raspberry pi で実行する方法(クロスコンパイル)について説明する。

Raspberry pi のブートの仕組みはまず SD カードから OS を読み込んで起動、USB 接続の HDD から OS を読み込んで起動、ネットワーク越しの HDD から OS を読み込んで起動する。

一般的な組み込み機器にはディスプレイやキーボードはついていない。組み込み機器の操作はシリアル通信で行うのが一般的である。シリアルコンソールとはシリアル通信を介してホストの画面やキーボードとターゲットを仲介する。

組み込み機器とは特定の目的のためだけに作られている専用機器：TV 等の家電などターゲットとホストがある。ターゲットとは組み込み解春における開発対象の組み込み機器のことである。ホストとはターゲット向けのソフト開発やコンパイルを行うためのコンピュータのことである。

ターゲットは raspberry pi であり、ホストは PC のことである。

以下では raspberru pi をラズパイと表す。

ホストとラズパイをシリアルケーブルで接続する。そして端末上で minicom を立ち上げるために `sudo minicom` と実行し、minicom を立ち上げる。Minicom を立ち上げたら、SD カードを挿し、LAN ケーブルを接続する。そしてラズパイを起動する。

立ち上がったらユーザー名 pi、パスワード raspberry とする。Ip アドレスを確認するために `ip a` を実行する。

簡単なプログラムを作り、これをターゲット用クロスコンパイルし、ターゲットに転送して実行する。PC 上の端末で `arm-linux-gnueabi-hf-gcc main.c` として ARM 用のコードを作る。

ここで `sudo systemctl enable ssh` をして、SSH サーバーを起動。Sudo `systemctl start ssh` で SSH を起動する。そしてコンパイルしたファイルを scp を使ってラズパイに投げる。

端末で `scp a.out`(コンパイルで作ったファイル) pi@172.28.34.107:としてラズパイにファイルをコピーする。Minicom で `./a.out` と実行する。このプログラムは 1 から 10 億まで足すプログラムであり、PC とラズパイで実行時間を比較した。PC での実行時間は 216171523ns、ターゲットは 9851017762ns であった。ラズパイのほうが 5 倍ほど実行時間が長いことが分かった。

Raspberry pi の I2C に接続されたデバイスを C を使って操作する。I2C は 3 線だけでデバイスを安価に接続できるバスで、センサーなどのデータ転送量の少ないデバイス接続に用いられる。ラズパイでは I2C に駅所ディスプレイ(LCD)と温湿度/気圧センサーが接続されている。I2C はマスター(PC)と複数のスレーブから構成される。スレーブにはアドレスが割り振られている。I2C で通信するにはマスターが最初にスレーブアドレスを送信し、続いて複数のデータを送受信する。LCD のアドレスは 0x3e である。I2C 市宇佐の基本手順は次である。

```
int i2c = open("/dev/i2c-1", O_RDWR);           // I2C デバイスを開く
ioctl(i2c, I2C_SLAVE, 0x3e);                    // スレーブアドレスを指定
initLCD(i2c);                                     // 初期化 (lcd.c で定義)
lcd_datawrite(i2c, "hello");                     // データを書き込む
close(i2c);                                       // デバイスを閉じる
```

LCD はコマンド送信により表示を制御している。例えば 0x01(Clear Display)は実行後 1ms の待機が必要である。

メモリマップド IO とは、CPU に接続されたハードウェアを制御する方法の 1 つである。メモリマップド IO ではハードの制御レジスタがメモリ内のある番地に置かれており、そこに読み書きをすることで制御をする。

GPIO とは Raspberry Pi の汎用入出力ピンである。タクトスイッチや LED の制御に利用する。

GPIO 制御の例

```
int fd = open("/dev/mem", O_RDWR);               // 物理メモリを開く
void *gpio_baseaddr = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                             MAP_SHARED, fd, GPIO_PHY_BASEADDR);
```

GPIO_PHY_BASEADDR = 0x3F200000 は GPIO の制御レジスタの物理アドレスである。
gpio_baseaddr を使って GPIO レジスタにアクセス可能にしている。

3. 動作の概要

○動作の概要

・構成要素

main：ソケット作成、接続処理、ファイル読み込み処理

bank：各銀行におけるトランザクション処理（スレッドの生成、送受信含む）

fun：受金専用スレッドでソケットからデータを受信し、該当口座に加算。

・スレッドの使い分け

送金処理はメインスレッド、受金処理はファンスレッド

・ソケット通信：

双方向通信のために 2 本の TCP ソケットを使用（送信用、受信用）

・連携の仕組み

.csv の各行は自行内の振替(t)と他行宛の送金(o)をトランザクションから判断

send/recv を用いた相手銀行との通信

○各関数と main 文について

・fun 関数：戻り値 void | 引数 void *arg 受金側の send で使うための sockfd である。

説明：fun 関数はスレッドの関数である。今回のプログラムにおいては相手方が振込処理を行う際の受金側の処理を fun 関数のスレッドで行っている。

・bank 関数：戻り値 void | 引数 int sock1,sock2, FILE* istream sock1 は受金用(recv 用 socket)、sock2 は送金用(send 用 socket)、istream はサーバーとクライアントそれぞれの.csv ファイルのファイルポインタ

説明：大きく 2 つの処理をしている。1.ファイルの読み込み、2.口座の送受信処理の 2 つである。

・sumAmount 関数：戻り値 int 全ての口座の残高の合計 | 引数 void

説明：処理終了後、すべての口座の残高を足し合わせた値を計算して戻り値として返す。

・main 文：引数 int argc,char* argv[] argc は使わない。argv にはオプションとしてサーバー側のプロセスでは s が入り、クライアント側のプロセスでは c が入る。

説明：大きく 3 つのセクションに分かれている。1.サーバー側のプロセスの処理、2.クライアント側のプロセスの処理、3.処理終了後の各講座の残高の合計を出力する処理の 3 つである。

○処理順序

- ①main 文で各プロセス（サーバー、クライアント）の通信処理
- ②bank 関数で csv ファイルを読み込む
- ③bank 関数でスレッド作成、プロセスで並行に銀行の振込、振替処理、送金処理、スレッドの終了
- ④fun 関数が受金を担う
- ⑤処理が終わったら、main 内で各口座の残高を出力
- ⑥sumAmount 関数を使ってすべての口座の合計残高を出力

4. システムの詳細

1. プログラムのすべて

プログラム 1

```
1 //transactionのinclude
2 #include <stdio.h> // perror, printf, FILE, fopen, fgetsで必要
3 #include <pthread.h> // threadを扱うためのinclude
4 #include <time.h> // 現在時刻を持ってくるためのinclude
5
6 //サーバー、クライアントのinclude
7 #include <arpa/inet.h> // sockaddr_in, inet_addr, htonsで使用
8 #include <string.h> // memsetで使用
9 #include <unistd.h> // write, connect, read, close, sleep で使用
10 #include <sys/socket.h> // socket, connectで使用
11 #include <netinet/in.h> // sockaddr_inで使用
12 #include <stdlib.h> // exitで使用
13 #include <errno.h> // errnoで使用
14
15 #define N 10000 // 処理の数
16 #define A 100 // 口座の数
17
18 //銀行設定
19 char to_bank[N] // 銀行指定
20 int from[N], to[N], amount[N] // csvの情報を入れる
21 int account[A] // 各口座の残高
22
23 //mutexの宣言
24 pthread_mutex_t mutex1;
25
26 void busy(int index) // 時間稼ぎ
27 {
28     for(int k=0; k<amount[index]*100; k++)
29         ;
30 }
31
32 void *fun(void *arg) // threadを実行する関数
33 {
34     int sockfd = (int) arg;
35     int data[2];
36     int ct = 1;
37     while(1){
38         ssize_t len = recv(sockfd, data, sizeof(data), 0);
39         // printf("%d:%d\n", ct, len);
40         if(len < 0){
41             perror("recv");
42             break;
43         }
44         else if(len == sizeof(data)){
45             pthread_mutex_lock(&mutex1);
46             // printf("受信: 口座%dに%d円加算\n", data[1], data[0]);
47             account[data[0]] += data[1];
48             pthread_mutex_unlock(&mutex1);
49         }
50         else if(len == 0){
51             break;
52         }
53         else{
54             printf("データをもらえてない\n");
55         }
56         ct++;
57     }
58     sleep(1);
59     close(sockfd);
60 }
```

プログラム 2

```

55 }
56
57
58
59 void bank(int sock1,int sock2, FILE* istream){//各銀行がやる処理
60
61     ////////////////////////////////////////ファイルの読み込み
62     int i = 0,val;
63     while(1){
64         val = fscanf(istream,"%d,%d,%d,%d\n",&to_bank[i],&from[i],&to[i],&amount[i]);
65         if (val== -1) break;
66         i++;
67     }
68     fclose(istream);
69
70
71     //mutexの初期化
72     pthread_mutex_init(&mutex1, NULL);
73     //条件変数の初期化
74     //pthread_cond_init(&cvar,NULL);
75
76     //スレッドを作る
77     pthread_t th0;
78     if(pthread_create(&th0,NULL,fun,(void *)sock1)!=0){//引数sock1(受金のためにthreadに送らなければいけない)
79         perror("pthread_create");
80         exit(1);
81     }
82
83     ////////////////////////////////////////口座の処理
84     int cur = 0;//現在処理しようとしている行
85     int num[2];//送るデータ
86     while(cur < N){//口座の数だけ調べる
87         if(to_bank[cur]==t){//振替
88             //printf("t\n");
89             pthread_mutex_lock(&mutex1);
90             account[from[cur]]-=amount[cur];
91             //printf("%d回目:口座%dから%d円減算",cur,from[cur],amount[cur]);
92             account[to[cur]]+=amount[cur];
93             //printf("口座%dから%d円加算\n",cur,to[cur],amount[cur]);
94             pthread_mutex_unlock(&mutex1);
95         }else{//振込
96             pthread_mutex_lock(&mutex1);
97             //printf("o\n");
98             account[from[cur]]-=amount[cur];
99             pthread_mutex_unlock(&mutex1);
100             //printf("%d回目:口座%dから%d円減算\n",cur,from[cur],amount[cur]);
101             num[0] = to[cur];//相手の口座番号
102             num[1] = amount[cur];//口座の金額
103
104             if(send(sock2,num,sizeof(num), 0)==-1){
105                 perror("send");
106                 close(sock2);
107                 exit(1);
108             }
109         }
110         //printf("%d\n",cur);
111         cur++;
112     }
113     close(sock2);
114     //スレッドの終了を待ち合わせる
115     pthread_join(th0,NULL);
116
117     return;
118 }
119

```

プログラム 3

```

118 }
119
120 int sumAmount()
121 {
122     int sum = 0;
123     for(int j=0;j<A;j++){
124         sum += account[j];
125     }
126     return sum;
127 }
128
129 int main(int argc, char *argv[]){
130     struct sockaddr_in serv_addr;
131     int sockfd,sockfd2,sock1,sock2;//sockfd:クライアント待つ用、sock1,2:送受金用
132     for(int i=0;i<A;i++){//まずすべての口座の残高は10000円とする
133         account[i]=10000;
134     }
135     //////////////////////////////////////通信
136     // ソケットを作る
137     if((sockfd = socket(AF_INET, SOCK_STREAM, 0))==-1){
138         perror("socket");//errorを使っている
139         exit(1);//プログラムがエラーで終了したことを示す
140     }
141     if((sockfd2 = socket(AF_INET, SOCK_STREAM, 0))==-1){
142         perror("socket");//errorを使っている
143         exit(1);//プログラムがエラーで終了したことを示す
144     }
145     printf("ソケット作成完了\n");
146
147     // サーバー、クライアントのアドレスを作る
148     memset(&serv_addr, 0, sizeof(struct sockaddr_in));
149     serv_addr.sin_family = AF_INET;
150     serv_addr.sin_port = htons(50000);//50000に設定
151     //////////////////////////////////////サーバーのみ、実行する関数
152     if(strcmp(argv[1], "s")==0){
153         FILE *istream1;
154         istream1 = fopen("trans2.csv","r");
155
156         serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
157         int flag = 1;
158         if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &flag, sizeof(int)) == -1){
159             perror("setsockopt");
160             exit(1);
161         }
162         // ソケットにアドレスを割り当てる
163         if (bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) == -1){
164             perror("bind");
165             close(sockfd);
166             exit(1);
167         }
168         // コネクション要求を待ち始めるよう指示
169         if (listen(sockfd, 5) == -1){
170             perror("listen");
171             close(sockfd);
172             exit(1);
173         }
174         // 要求があったらそれを受け付ける (なければ待つ)
175         if ((sock1 = accept(sockfd, NULL, NULL)) == -1){
176             perror("accept1");
177             close(sockfd);
178             exit(1);
179         }
180         if ((sock2 = accept(sockfd, NULL, NULL)) == -1){
181             perror("accept2");
182             close(sockfd);
183             exit(1);
184         }
185     }
186     printf("サーバー準備完了\n");

```

プログラム 4

```

172     }
173     // 要求があったらそれを受け付ける (なければ待つ)
174     if ((sock1 = accept(sockfd, NULL, NULL)) == -1) {
175         perror("accept1");
176         close(sockfd);
177         exit(1);
178     }
179     if ((sock2 = accept(sockfd, NULL, NULL)) == -1) {
180         perror("accept2");
181         close(sockfd);
182         exit(1);
183     }
184     printf("サーバー側準備完了\n");
185     //サーバー側の処理↓
186     bank(sock2, sock1, istream1);
187 }
188 //////////////////////////////////////クライアントのみ実行する関数
189 if (strcmp(argv[1], "c") == 0) {
190     FILE *istream2;
191     istream2 = fopen("trans3.csv", "r");
192
193     serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1"); //自分自身
194
195
196     sleep(1); //サーバーの起動を待つ
197     // コネクションを張るための要求をサーバーに送る
198     if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr_in)) == -1) {
199         perror("connect1"); //errorを使っている
200         close(sockfd);
201         exit(1); //プログラムがエラーで終了したことを示す
202     }
203     if (connect(sockfd2, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr_in)) == -1) {
204         perror("connect1"); //errorを使っている
205         close(sockfd);
206         exit(1); //プログラムがエラーで終了したことを示す
207     }
208     printf("クライアント側準備完了\n");
209     //クライアント側の処理
210     bank(sockfd, sockfd2, istream2);
211 }
212
213
214 //各口座の残高
215 printf("各口座の残高\n");
216 for (int k = 0; k < A; k++) {
217     printf("口座%d: %d\n", k + 1, account[k]);
218 }
219
220 printf("現在のすべての講座の総残高: %d\n", sumAmount());
221
222
223

```

2. プログラムの詳細な説明

・ txgen について

本体のプログラムで使う.csv を作成するためのプログラム txgen.c を作成した。

```
1  #include <time.h>
2  #include <stdlib.h>
3  #include <stdio.h>
4
5  int main(int argc, char *argv[]){
6      int num_txn, num_account, max_amount, i, op;
7      if (argc!=4) {
8          printf("usage: txgen num_txn num_account max_amount\n");
9          exit(1);
10     }
11     num_txn = atoi(argv[1]);
12     num_account = atoi(argv[2]);
13     max_amount = atoi(argv[3]);
14
15     srand(time(NULL));
16
17     // printf("num_txn=%d, num_account=%d, max_amount=%d\n", num_txn, num_account, max_amount);
18     for (i = 0; i < num_txn; i++) {
19         int toVal, amount;
20         int fromVal = rand() % num_account;
21         while (1) {
22             toVal = rand() % num_account;
23             if (fromVal != toVal) break;
24         }
25         amount = (rand() % max_amount) + 1;
26         op = rand() % 2;
27         if(op == 0){
28             printf("t,%d,%d,%d\n", fromVal, toVal, amount);
29         }
30         if(op == 1){
31             printf("o,%d,%d,%d\n", fromVal, toVal, amount);
32         }
33     }
34     return 0;
35 }
36
```

cc -pthread txgen.c -o txgen

./txgen 10000 100 100 > trans.csv

上記のコマンドで実行する。なお、実行時のプログラムへの引数の意味は左から（取引数、口座数、最大金額となる。

trans.csv を開くと次のようなデータが出力されている。

t,3,18,45→口座番号 3 から口座番号 18 に 45 円振替処理を行うことを意味する。

o,5,7,91→自分の口座番号 5 から他行の口座番号 7 に 91 円振替処理を行うことを意味する。

・通信部分について

以下ではサーバー側のプロセスを s、クライアント側のプロセスを c と表す。

◇s,c の共通部分

```
134 //////////////////////////////////////////////////通信
135 // ソケットを作る
136 if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1){
137     perror("socket");//errorを使っている
138     exit(1);//プログラムがエラーで終了したことを示す
139 }
140 if((sockfd2 = socket(AF_INET, SOCK_STREAM, 0)) == -1){
141     perror("socket");//errorを使っている
142     exit(1);//プログラムがエラーで終了したことを示す
143 }
144 printf("ソケット作成完了\n");
145
146 // サーバー、クライアントのアドレスを作る
147 memset(&serv_addr, 0, sizeof(struct sockaddr_in));
148 serv_addr.sin_family = AF_INET;
149 serv_addr.sin_port = htons(50000);//50000に設定
```

まず通信するために socket 関数を用いてソケットを 2 つ作る。ここで 2 つ作る理由はこの後にサーバー内のプログラムで出てくる connect 関数を使う際に connect の引数として 2 つのソケットが必要だからである。

また、サーバーとクライアント両方でアドレスを作る。自由に使えるポート番号 50000 を割り当てる。

◇sのみ行う部分

```
150 //////////////////////////////////////////////////サーバーのみ、実行する関数
151 if(strcmp(argv[1], "s") == 0){
152     FILE *istream1;
153     istream1 = fopen("trans2.csv", "r");
154
155     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
156     int flag = 1;
157     if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &flag, sizeof(int)) == -1) {
158         perror("setsockopt");
159         exit(1);
160     }
161     // ソケットにアドレスを割り当てる
162     if (bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) == -1) {
163         perror("bind");
164         close(sockfd);
165         exit(1);
166     }
167     // コネクション要求を待ち始めるよう指示
168     if (listen(sockfd, 5) == -1) {
169         perror("listen");
170         close(sockfd);
171         exit(1);
172     }
173     // 要求があったらそれを受け付ける (なければ待つ)
174     if ((sock1 = accept(sockfd, NULL, NULL)) == -1) {
175         perror("accept1");
176         close(sockfd);
177         exit(1);
178     }
179     if ((sock2 = accept(sockfd, NULL, NULL)) == -1) {
180         perror("accept2");
181         close(sockfd);
182         exit(1);
183     }
184     printf("サーバー側準備完了\n");
185     //サーバー側の処理↓
186     bank(sock2, sock1, istream1);
187 }
```

サーバー側が読み込むファイルを trans2.csv として fopen を使って読み込んでいる。

serv_addr.sin_addr.s_addr = htonl(INADDR_ANY)を用いてサーバー側が任意のインターフェース（自分の持つすべての IP アドレス）で接続を待ち受けるようにしている。

flag と setsocket 関数を用いて、sockfd が使用中であっても構わず次の bind 関数をさせるために入れている。

その後の bind と listen について書いてある通りである。

sockfd は listen する際に使われるソケットであり、もしクライアント側から要求があればそのソケットとは別のソケットで通信を行う用になる。ここでは sock1, sock2 という 2 つのソケットが通信を行うソケットとなる。

これでサーバー側の通信準備は整う。

bank 関数の引数は 3.で説明した通り bank（受金用ソケット,送金用ソケット,そのプロセスで扱うファイルポインタ）である。そのため、サーバーにおいては sock2 を受金用、sock1 を送金用のソケットとしている。ファイルは trans2.csv を読み込んだ際に代入したファイルポインタを引数としている。

◇cのみ行う部分

```
////////////////////////////////////クライアントのみ実行する関数
if(strcmp(argv[1], "c") == 0){
    FILE *istream2;
    istream2 = fopen("trans3.csv", "r");

    serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");//自分自身

    sleep(1);//サーバーの起動を待つ
    // コネクションを張るための要求をサーバに送る
    if(connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr_in)) == -1){
        perror("connect1");//errorを使っている
        close(sockfd);
        exit(1);//プログラムがエラーで終了したことを示す
    }
    if(connect(sockfd2, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr_in)) == -1){
        perror("connect1");//errorを使っている
        close(sockfd);
        exit(1);//プログラムがエラーで終了したことを示す
    }
    printf("クライアント側準備完了\n");
    //クライアント側の処理
    bank(sockfd, sockfd2, istream2);
}
```

まず、ファイルについてはサーバー同様に open 関数を使ってクライアント側では trans3.csv を読み込んでいます。

serv_addr.sin_addr.s_addr = htonl(127.0.0.1)で自分自身へ通信を行う用にしています。

connect 関数では作っておいた 2 つのソケット、sockfd と sockfd2 を使って要求を送信している。

これでクライアント側の通信準備は整う。

サーバー同様、クライアントにおいては sockfd を受金用、sockfd2 を送金用のソケットとしている。ファイルは trans3.csv を読み込んだ際に代入したファイルポインタを引数としている。

・ bank 関数について

```
58
59 void bank(int sock1,int sock2, FILE* istream){//各銀行がやる処理
60
61     ////////////////////////////////////////////////////ファイルの読み込み
62     int i = 0, val;
63     while(1){
64         val = fscanf(istream, "%d,%d,%d,%d\n", &to_bank[i], &from[i], &to[i], &amount[i]);
65         if (val == -1) break;
66         i++;
67     }
68     fclose(istream);
69
70
71     //mutexの初期化
72     pthread_mutex_init(&mutex1, NULL);
73     //条件変数の初期化
74     //pthread_cond_init(&cover, NULL);
75
76     //スレッドを作る
77     pthread_t th0;
78     if(pthread_create(&th0, NULL, fun, (void *)sock1) != 0){//引数sock1(受金のためにthreadに送らなければいけない)
79         perror("pthread_create");
80         exit(1);
81     }
82
83     ////////////////////////////////////////////口座の処理
84     int cur = 0; //現在処理しようとしている行
85     int num[2]; //送るデータ
86     while(cur < N){ //口座の数だけ調べる
87         if(to_bank[cur] == 't'){ //振替
88             //printf("t\n");
89             pthread_mutex_lock(&mutex1);
90             account[from[cur]] -= amount[cur];
91             //printf("%d回目:口座%dから%d円減算", cur, from[cur], amount[cur]);
92             account[to[cur]] += amount[cur];
93             //printf("口座%dから%d円加算\n", cur, to[cur], amount[cur]);
94             pthread_mutex_unlock(&mutex1);
95         } else { //振込
96             pthread_mutex_lock(&mutex1);
97             //printf("o\n");
98             account[from[cur]] -= amount[cur];
99             pthread_mutex_unlock(&mutex1);
100             //printf("%d回目:口座%dから%d円減算\n", cur, from[cur], amount[cur]);
101             num[0] = to[cur]; //相手の口座番号
102             num[1] = amount[cur]; //口座の金額
103
104             if(send(sock2, num, sizeof(num), 0) == -1){
105                 perror("send");
106                 close(sock2);
107                 exit(1);
108             }
109         }
110         //printf("%d\n", cur);
111         cur++;
112     }
113     close(sock2);
114     //スレッドの終了を待ち合わせる
115     pthread_join(th0, NULL);
116
117     return;
118 }
```

bank 関数は二つのセクションに分かれており、ファイルを読み込む処理と銀行の振替、振込処理を行う。

引数として送られてきたファイルポインタを用いてファイルのデータを読み込んでグローバル変数として宣言しておいた配列 to_bank, from, to, amount に読み込んだデータを代入する。bank 関数内でスレッドの作成を行う。ここで pthread_create の引数として sock1、つまり受金側のソケットをスレッドの引数として与えている。今回スレッドが担うのは受金処理であるため、受信用である sock1 を引数としたのである。

次に銀行の処理に移る。while を用いてトランザクションの回数だけ繰り返すようにする。

to_bank に't'が入っているなら、自分の銀行内での振替処理であるため from に入っている口座番号の口座の残高から amount 円減算し、to に入っている口座番号の残高に加算する。to_bank に'o'が入っているなら、他行宛に振込の処理をしなければいけない。まず自分の銀行内で from に入っている口座番号の口座の残高から amount を減算する。そのあと他行が必要なデータである2つのデータを配列に入れる。1つ(num[0])は相手の何番の口座番号、もう1つはいくら加算するか金額(num[1])である。ここで使っている send 関数で引数に sock2 を採用している。これは bank 関数の引数で送金側に相当するソケットである。サーバー側では sock1、クライアント側では sockfd2 を送金用に引数としていた。

・ fun 関数

```
29
30 void *fun(void *arg){/threadを実行する関数
31
32     int sockfd = (int)arg;
33     int data[2];
34     int ct = 1;
35     while(1){
36         ssize_t len = recv(sockfd,data,sizeof(data), 0);
37         //printf(" %d:%d¥n",ct,len);
38         if(len < 0){
39             perror("recv");
40             break;
41         }else if(len == sizeof(data)){
42             pthread_mutex_lock(&mutex1);
43             //printf("受信:口座%dに%d円加算-----",data[0],data[1]);
44             account[data[0]]+=data[1];
45             pthread_mutex_unlock(&mutex1);
46         }else if(len == 0){
47             break;
48         }else{
49             printf("データをもらえてない¥n");
50         }
51         ct++;
52     }
53     sleep(1);
54     close(sockfd);
55
56 }
57
```

引数は受金用ソケットである。While は無限ループしている。recv 関数を使って、相手のスレッドから送られてきたデータを読み込んでいる。引数として送られてきた受金用ソケットを入れている。これはサーバー側では sock2、クライアント側では sockfd1 を受金用に引数としていた。送られてきたデータは配列 data に格納する。num[0]が data[0]に、num[1]が data[1]に格納されている。recv の返り値は正常に動作した場合、受信したバイト数、エラーの時-1、ソケットの接続相手が正しくシャットダウンを実行した場合は 0 を返す。相手の接続が切れたらソケットは必要ないため close している。

3. テスト、動作確認

① 実行後、すべての口座の残高が 200 万円になっているか確認する。

csv のプログラムへの引数を 10000(トランザクション数),100(口座数),100(最大金額)として実行。下に添付した画像は左がサーバー、右がクライアントを実行している。また trans2.csv と trans3.csv を変えて何回か実行し、最終的な 2 つの銀行の口座の残高の合計が 200 万になっているか調べた。

```
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 1003186
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 999530
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 999530
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 999530
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 999530
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 994827
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 994827
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
現在のすべての講座の総残高: 994827
cy23263@pj-linux-7:~/el$

cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 996814
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 996814
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 996814
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 996814
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 996814
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 996814
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 1000470
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 1000470
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 1000470
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 1005173
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 1005173
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
現在のすべての講座の総残高: 1005173
cy23263@pj-linux-7:~/el$
```

写真を見ると 1 回目でサーバー側 1003186、クライアント側 996814 となっており、合計すると 200 万となる。

2 回目でサーバー側 999530、クライアント側 1000470 となっており、合計すると 200 万となる。

3 回目でサーバー側 994827、クライアント側 1005173 となっており、合計すると 200 万となる。

よってプログラムの実行前と実行後ですべての口座の残高は 200 万で一致することとなっている。消えた金額はなしということが確認できた。

② 送金、受金がうまく出来ているか確認

trans.csv のプログラムへの引数を 3(トランザクション数),10(口座数),10(最大金額)として実行する。読み込んだファイルの内容と送信時の加算、減算の内容、受信時の加算の内容を出力することで送受信の内容を確認する。

```
cy23263@pj-linux-7:~/el$ ./newtrans s
ソケット作成完了
サーバー側準備完了
to_bank[0]=o, from[0]=7, to[0]=9, amount[0]=4
to_bank[1]=t, from[1]=5, to[1]=2, amount[1]=5
to_bank[2]=t, from[2]=3, to[2]=4, amount[2]=6
o : 1 回目 : 口座 7 から 4 円減算
受信データ : 口座番号=8, 金額=2
受信データ : 口座番号=6, 金額=4
t : 2 回目 : 口座 5 から 5 円減算 口座 2 に 5 円加算
t : 3 回目 : 口座 3 から 6 円減算 口座 4 に 6 円加算
口座 0 : 10000
口座 1 : 10000
口座 2 : 10005
口座 3 : 9994
口座 4 : 10006
口座 5 : 9995
口座 6 : 10004
口座 7 : 9996
口座 8 : 10002
口座 9 : 10000
現在のすべての講座の総残高 : 100002
```

サーバー側の実行結果を表示している。

ファイルで読み込まれたデータは左の通りである。

振替処理は t、振込処理は o で表示され、受信データがクライアント側から来たデータである。

最終的な残高と銀行の処理内容が一致しているか確認する。なお、受信データはその口座番号の金額だけ加算することを表している。

```
cy23263@pj-linux-7:~/el$ ./newtrans c
ソケット作成完了
クライアント側準備完了
to_bank[0]=t,    from[0]=7,    to[0]=6,
amount[0]=10
to_bank[1]=o,    from[1]=5,    to[1]=8,
amount[1]=2
to_bank[2]=o,    from[2]=2,    to[2]=6,
amount[2]=4
t: 1 回目: 口座 7 から 10 円減算 口座 6 に
10 円加算
o: 2 回目: 口座 5 から 2 円減算
o: 3 回目: 口座 2 から 4 円減算
受信データ: 口座番号=9, 金額=4
口座 0: 10000
口座 1: 10000
口座 2: 9996
口座 3: 10000
口座 4: 10000
口座 5: 9998
口座 6: 10010
口座 7: 9990
口座 8: 10000
口座 9: 10004
現在のすべての講座の総残高: 99998
```

左はクライアント側の出力結果である。サーバーと同じように一致しているか確認してすべて一致していた。よってクライアント側の銀行処理が正常に動作していることを確認できた。

③ エラーテスト

- ・ bind のエラー

2つの端末でサーバーのプロセスのプログラムを実行する。

```
cy23263@pj-linux-7:~/el$ ./newtrans s  
ソケット作成完了  
bind: Address already in use
```

左のように表示された。bind のエラー判定はうまく機能していることが確認できた。

- ・ accept のエラー

sockfd が accept に入る直前に sockfd に 1 を代入し、引数とした。

```
cy23263@pj-linux-7:~/el$ ./newtrans s  
ソケット作成完了  
Accept: Socket operation on non-socket
```

左のように表示された。accept のエラー判定はうまく機能していることが確認できた。

- ・ connect のエラー

クライアントのプロセスを先に実行する。

```
cy23263@pj-linux-7:~/el$ ./newtrans c  
ソケット作成完了  
connect:Connection refused
```

左のように表示された。connect のエラー判定はうまく機能していることが確認できた。

- ・ recv のエラー

sockfd が recv に入る直前に sockfd に 1 を代入し、引数とした。

```
cy23263@pj-linux-7:~/el$ ./newtrans s  
ソケット作成完了  
サーバー側準備完了  
recv: Socket operation on non-socket
```

左のように表示された。recv のエラー判定はうまく機能していることが確認できた。

5. 終わりに

以上、プロジェクト演習 14 の最終課題である銀行間の並行プログラミングの内容について説明した。

感想：本演習を通して、並行プログラミングとネットワーク通信の基本を体系的に学ぶことができた。特に、スレッド間で共有するデータの整合性を保つための mutex 制御や、ソケット通信におけるバッファ管理の重要性を実感した。また、通信トラブル時のエラー処理や、スレッドの動機タイミングによっては予期せぬ動作を引き起こす可能性があることを知り、堅牢な設計の難しさを学んだ。

時間があれば拡張機能にも手を出したかった。もっと研鑽に励む。